Metaheurísticas

Aula 8 — Tópicos Avançados e Perspectivas

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula

Ao final desta aula, o aluno deve ser capaz de:

1. Aplicar métodos automáticos de ajuste de hiperparâmetros (grid search, random search, Otimização Bayesiana) e justificar a escolha entre eles com base no custo de avaliação;
2. Usar técnicas de análise de paisagem (FDC, autocorrelação, LON) como ferramentas de diagnóstico antes de escolher um algoritmo;
3. Identificar classes de problemas para as quais métodos exatos ou heurísticas construtivas são preferíveis a metaheurísticas;
4. Aplicar metaheurísticas à otimização de hiperparâmetros e arquitetura de redes neurais (neuroevolução), comparando com abordagens baseadas em gradiente;
5. Posicionar o campo das metaheurísticas no contexto atual de aprendizado de máquina, identificando oportunidades de integração.

1 Ajuste de Hiperparâmetros

Todo algoritmo estudado neste curso tem parâmetros que precisam ser configurados: temperatura inicial do SA, tenure da BT, F e C_R da DE, ω e coeficientes do PSO, N_p e p_m do AG. A calibração desses **hiperparâmetros** é frequentemente o gargalo prático que separa uma implementação medíocre de uma competitiva.

O problema de ajuste de hiperparâmetros é ele mesmo um problema de otimização: dado um espaço de configurações Θ e uma métrica de desempenho $\mathcal{Q}(\theta)$ (e.g., custo médio em 30 execuções), busca-se $\theta^* = \arg\min_{\theta\in\Theta} \mathcal{Q}(\theta)$. O desafio é que cada avaliação de $\mathcal{Q}(\theta)$ requer executar o algoritmo completo — potencialmente caro.

1.1 Grid Search

O método mais simples: define-se uma grade discreta de valores para cada hiperparâmetro e avalia-se $\mathcal{Q}$ em todos os pontos da grade.

Vantagem: exaustivo na grade — garante que a melhor combinação na grade seja encontrada.

Desvantagem: cresce exponencialmente com o número de hiperparâmetros. Para 4 parâmetros com 5 valores cada: $5^4 = 625$ avaliações, cada uma custando 30 execuções do algoritmo — 18 750 execuções totais.

Quando usar: apenas para 1–2 hiperparâmetros com faixas bem conhecidas.

1.2 Random Search

Bergstra & Bengio (2012) demonstraram que amostrar aleatoriamente o espaço Θ é sistematicamente mais eficiente que grid search quando apenas alguns parâmetros importam de fato. A intuição: em uma grade 5×5, se apenas o primeiro parâmetro importa, grid search testa apenas 5 valores efetivos desse parâmetro enquanto random search testa 25 valores distintos.

Regra prática: com orçamento de B avaliações, random search cobre o espaço de forma mais informativa que grid search para B menor que o tamanho total da grade.

1.3 Otimização Bayesiana

A **Otimização Bayesiana** (BO) constrói um **modelo substituto** (*surrogate*) da função $\mathcal{Q}(\theta)$ — tipicamente um **Processo Gaussiano** (GP) — e usa esse modelo para selecionar o próximo ponto a avaliar de forma informada. O ciclo é:

1. Avalia $\mathcal{Q}(\theta)$ em alguns pontos iniciais (e.g., aleatórios).
2. Ajusta o GP aos dados observados.
3. Maximiza uma **função de aquisição** para selecionar o próximo θ. A *Expected Improvement* (EI) é a mais comum:

$$\mathrm{EI}(\theta) = \mathbb{E}[\max(0, \mathcal{Q}^* - \mathcal{Q}(\theta))]$$

4. Avalia $\mathcal{Q}(\theta_{\text{novo}})$ e atualiza o GP. Repete até o orçamento se esgotar.

A BO é particularmente valiosa quando cada avaliação de $\mathcal{Q}$ é cara. Bibliotecas como `scikit-optimize`, `Optuna` e `SMAC3` implementam BO de forma pronta para uso.

Exemplo

Calibração automática do SA para o TSP com $n = 30$ cidades (instância do fio condutor). Hiperparâmetros: $T_0 \in [100, 5000]$, $\alpha \in [0.88, 0.99]$, $L \in [50, 500]$ (3 dimensões contínuas). Métrica $\mathcal{Q}(\theta)$: comprimento médio do melhor tour em 30 execuções.

Método	Melhor $\mathcal{Q}$ (km)	Avaliações usadas	Observações
Manual (regra de taxa aceite.)	851	5	Requer expertise; não reprodutível
Grid Search 5^3	839	125	Impraticável com orçamento de 50
Random Search	826	50	Simples; boa cobertura
Otimização Bayesiana (GP-EI)	812	50	Melhor resultado; aprende estrutura

Table 1: Comparação de métodos de calibração de hiperparâmetros do SA para o TSP $n = 30$.

A BO encontrou $T_0 = 3,840$, $\alpha = 0.971$, $L = 280$ — configuração que seria difícil de identificar manualmente e que a grade 5^3 sequer cobriria com o orçamento de 50 avaliações. Note que a BO usa o mesmo princípio de *surrogate* da Seção 6 — aqui aplicado ao espaço de hiperparâmetros, não ao espaço de soluções.

2 Análise de Paisagem de Busca

Antes de escolher um algoritmo, o engenheiro deveria — idealmente — entender a estrutura da paisagem do problema. Técnicas de **análise de paisagem** fornecem informação diagnóstica que orienta essa escolha sem executar qualquer algoritmo de otimização completo.

2.1 Fitness-Distance Correlation (FDC)

A **FDC** mede a correlação entre o valor da função objetivo e a distância ao ótimo global conhecido:

$$\mathrm{FDC} = \frac{\sum_i \left(f_i - \overline{f}\right)\left(d_i - \overline{d}\right)}{\sqrt{\sum_i \left(f_i - \overline{f}\right)^2 \sum_i \left(d_i - \overline{d}\right)^2}}$$

onde $f_i = f(\boldsymbol{x}_i)$ e $d_i = d(\boldsymbol{x}_i, \boldsymbol{x}^*)$ é a distância ao ótimo.

- $\mathrm{FDC} < -0.15$: paisagem **benevolente** — soluções melhores tendem a estar mais próximas do ótimo; busca local funciona bem.
- $|\mathrm{FDC}| \leq 0.15$: paisagem **difícil** — sem correlação clara; algoritmos populacionais com alta exploração são preferidos.
- $\mathrm{FDC} > 0.15$: paisagem **enganosa** (deceptive) — soluções melhores tendem a estar **mais longe** do ótimo; algoritmos que seguem o gradiente são sistematicamente enganados.

2.2 Autocorrelação e comprimento de correlação

A **autocorrelação** da paisagem mede a correlação entre valores de f ao longo de uma caminhada aleatória:

$$r(s) = \frac{\mathbb{E}\left[(f(\boldsymbol{x}_t) - \mu)(f(\boldsymbol{x}_{t+s}) - \mu)\right]}{\sigma^2}$$

O **comprimento de correlação** ℓ é o lag em que $r(s)$ decai para $\frac{1}{e}$:

$$\ell = -\frac{1}{\ln(r(1))}$$

- ℓ grande: paisagem **suave** — vizinhos têm valores semelhantes; busca local funciona bem.
- ℓ pequeno: paisagem **rugosa** — valores mudam abruptamente; busca local tende a ficar presa; algoritmos populacionais são preferidos.

2.3 Local Optima Networks (LON)

Local Optima Networks (LON), propostos por Ochoa et al. (2008), representam a estrutura macro da paisagem como um grafo:

- **Nós**: ótimos locais (identificados por Hill Climbing a partir de múltiplos pontos de partida).
- **Arestas**: transições de escape entre ótimos locais — a probabilidade de ir do ótimo A ao ótimo B após uma perturbação.

A análise do LON revela o número de ótimos locais, o tamanho das bacias de atração, a presença de grandes bacias ao redor do ótimo global, e caminhos de escape entre ótimos — informação que orienta o design de operadores de perturbação no SA e BT.

Observação
A análise de paisagem é computacionalmente cara: calcular a FDC requer conhecer o ótimo; construir o LON requer centenas a milhares de execuções de HC. A abordagem prática é amostrar a paisagem em versões **simplificadas** do problema e extrapolar as conclusões — mas com cautela, pois a estrutura da paisagem pode mudar com a escala.

3 Quando Não Usar Metaheurísticas

Metaheurísticas são ferramentas poderosas, mas não universais. Há classes de problemas onde outros métodos são claramente superiores.

3.1 Problemas convexos

Se a função objetivo e as restrições são convexas, **métodos de pontos interiores** ou **gradiente projetado** encontram o ótimo global com garantia e eficiência computacional muito superior a qualquer metaheurística. Problemas lineares, quadráticos convexos e programação semidefinida têm solvers maduros (CPLEX, Gurobi, MOSEK) que devem ser a primeira escolha.

Regra: sempre verificar convexidade antes de recorrer a metaheurísticas. Se a Hessiana da função objetivo for positiva semidefinida e as restrições formarem um conjunto convexo, use um solver exato.

3.2 Problemas de pequena escala

Para problemas com espaço de busca pequeno ($|\mathcal{F}| < 10^4$), **enumeração completa** ou **branch-and-bound** encontram o ótimo global com certeza e frequentemente em tempo menor que 30 execuções de uma metaheurística. O Knapsack com $n \leq 20$ itens, por exemplo, é resolvido exatamente por programação dinâmica em $O(nW)$ — mais rápido e mais confiável que qualquer metaheurística.

3.3 Problemas com estrutura especial

Algumas classes têm algoritmos exatos eficientes:

- **Fluxo em redes**: mínimo custo de fluxo tem solução polinomial pelo simplex de redes.
- **Matching**: alocação ótima com custo linear tem solução pelo algoritmo húngaro em $O(n^3)$.
- **TSP com estrutura métrica e n pequeno**: programação dinâmica de Held-Karp em $O(2^n n^2)$ é exata para $n \leq 20$.
- **Problemas com subestrutura ótima**: programação dinâmica (mochila 0-1 com n e W moderados).

⚠️ **Atenção**
O custo de não questionar. O problema mais comum na aplicação de metaheurísticas não é escolher o algoritmo errado — é aplicar metaheurística em um problema que tem solução exata disponível. Antes de implementar qualquer metaheurística, responda: (1) o problema é convexo? (2) o espaço de busca é pequeno o suficiente para enumeração ou DP? (3) existe algoritmo de propósito específico para essa estrutura? Se qualquer resposta for "sim", o método exato é preferível.

3.4 Quando o tempo real não permite

Metaheurísticas com $N_p = 50$ e $T_{\max} = 200$ gerações requerem 10^4 avaliações de f. Se cada avaliação é cara (simulação física, inferência de modelo grande), o tempo total pode ser inviável para aplicações em tempo real. Nesses casos, considerar:

- **Modelos substitutos**: treinar uma rede neural offline que aproxima f com custo de avaliação $\approx 1\mu$ s, usar a metaheurística sobre o surrogate, e validar a solução com o modelo completo.
- **Heurísticas construtivas rápidas**: solução de qualidade conhecida em tempo garantido.
- **Otimização paramétrica offline**: resolver uma família de instâncias parametrizadas offline e consultar a tabela em tempo real.

4 Metaheurísticas e Aprendizado de Máquina

O campo das metaheurísticas está em rápida transformação pela integração com técnicas de aprendizado de máquina. Três direções são relevantes:

4.1 Aprendizado por reforço para seleção de operadores

Em vez de parâmetros fixos, algoritmos recentes usam **aprendizado por reforço** (RL) para selecionar dinamicamente qual operador aplicar (e.g., DE/rand vs. DE/best) com base no estado corrente da busca. O agente de RL aprende, ao longo das execuções, qual operador é mais eficaz em cada fase — diferenciando algoritmos **adaptativos** (como JADE) de algoritmos com **autoaprendizado** verdadeiro.

4.2 Modelos substitutos baseados em redes neurais

Para problemas onde f é cara (simulação física, renderização, avaliação de modelos grandes), redes neurais treinadas em avaliações anteriores substituem o simulador durante a maior parte da busca. A integração típica: AG ou PSO operam sobre o surrogate (avaliação rápida), com verificações periódicas no simulador real para atualizar o surrogate com novas amostras informativas. Esse paradigma — **surrogate-assisted optimization** — reduz o número de avaliações reais de 10^4 para 10^2–10^3 sem perda significativa de qualidade.

4.3 Estimation of Distribution Algorithms (EDA)

EDAs substituem os operadores de cruzamento e mutação por um **modelo probabilístico** do espaço de busca, aprendido a partir das melhores soluções da geração atual. A cada geração, novas soluções são **amostradas** do modelo em vez de geradas por cruzamento. Isso é a generalização extrema da DE: em vez de aprender a perturbação a partir de diferenças entre indivíduos, aprende-se a distribuição inteira das boas soluções. Em problemas com estrutura de dependência forte entre variáveis, EDAs que modelam correlações superaram AGs clássicos em qualidade de solução.

5 Neuroevolução: Otimizando Redes Neurais com Metaheurísticas

Uma das aplicações mais férteis e atuais das metaheurísticas é a otimização de redes neurais — tanto dos seus **hiperparâmetros** (taxa de aprendizado, número de camadas, dropout) quanto da sua própria **arquitetura** (topologia da rede, tipos de camadas, conexões). Esse campo é conhecido como

neuroevolução quando usa algoritmos evolutivos, e como **Neural Architecture Search** (NAS) na literatura de aprendizado de máquina.

5.1 Por que metaheurísticas para redes neurais?

O treinamento padrão de uma rede neural — retropropagação com gradiente estocástico (SGD/Adam) — otimiza os **pesos** $\boldsymbol{W}$ da rede com eficiência notável. Mas há dois problemas que o gradiente não resolve:

Otimização de hiperparâmetros: a taxa de aprendizado η, o esquema de decaimento, o tamanho do batch, os coeficientes de regularização, o dropout — são escolhidos antes do treinamento e não são diferenciáveis em relação à perda final. Formam um espaço de busca misto (contínuo e discreto) de alta dimensão, exatamente o domínio das metaheurísticas.

Otimização de arquitetura (NAS): o número de camadas, o número de neurônios por camada, o tipo de conexão (densa, convolucional, residual, atencional), a função de ativação — definem um espaço combinatório enorme. Uma rede com 10 camadas e 5 opções de tipo por camada tem $5^{\{10\}} \approx 10^7$ arquiteturas candidatas. Gradiente não se aplica diretamente a decisões discretas de topologia.

5.2 Representação de arquiteturas

A representação da arquitetura é o componente mais crítico. Três abordagens dominam:

Representação direta (lista de camadas). A arquitetura é codificada como uma sequência de decisões: tipo de camada, tamanho, função de ativação. Um cromossomo pode ser $[(\text{Conv}, 64, \text{ReLU}), (\text{Pool}, 2), (\text{Dense}, 128, \text{ReLU}), (\text{Dense}, 10, \text{Softmax})]$. Simples e interpretável, mas não captura conexões skip ou ramificações.

Representação baseada em grafo (NEAT e variantes). A rede é um grafo dirigido onde nós são neurônios e arestas são conexões com pesos. O algoritmo **NEAT** (NeuroEvolution of Augmenting Topologies, Stanley & Miikkulainen, 2002) evolui simultaneamente pesos e topologia, usando especiação para proteger inovações topológicas de extinção prematura.

Representação de célula (NAS baseado em blocos). Em vez de projetar a rede inteira, projeta-se um **bloco arquitetural** (célula) que é repetido. O espaço de busca se reduz drasticamente — de 10^7 para 10^3–10^4 arquiteturas candidatas de células. Esse é o paradigma do DARTS e do ENAS.

5.3 Algoritmo: AG para NAS com representação direta

Algoritmo — AG para Neural Architecture Search

```
NAS-AG(dados_treino, dados_val, N_pop, T_max, p_c, p_m):
  // inicialização: arquiteturas aleatórias
  para i = 1 até N_pop:
    arch_i ← gerar_arquitetura_aleatoria()  // sequência de camadas

  para t = 1 até T_max:
    // avaliação: treina cada rede e mede desempenho na validação
    para i = 1 até N_pop:
      modelo_i ← construir_rede(arch_i)
      treinar(modelo_i, dados_treino, epochs=E_proxy)  // orçamento reduzido
      fitness_i ← acuracia(modelo_i, dados_val)
```

```
    // seleção, cruzamento, mutação
    pais ← selecao_torneio(pop, fitness, k=2)
    filhos ← []
    para par ∈ pais em pares:
      filho ← cruzamento_um_ponto(par)        // troca sufixo de camadas
      filho ← mutacao(filho, p_m)             // adiciona/remove/altera camada
      filhos ← filhos + [filho]
    pop ← elitismo(pop, filhos, N_pop)

  arch_final ← argmax_i fitness_i
  // retreinamento completo da melhor arquitetura
  modelo_final ← construir_rede(arch_final)
  treinar(modelo_final, dados_treino, epochs=E_full)
  retorna modelo_final
```

O passo crítico é o **treinamento proxy** com `epochs=E_proxy` $\ll$ `E_full`: treinar cada candidata por completo seria proibitivo ($N_{\text{pop}} \times T_{\max}$ treinamentos completos). O proxy usa um orçamento reduzido de épocas como estimativa de qualidade da arquitetura — assumindo que a **ordenação relativa** das arquiteturas se preserva entre o treino curto e o treino completo. Isso é uma hipótese (frequentemente satisfeita, mas não garantida) e é a principal fonte de erro do AG para NAS.

5.4 PSO para otimização de hiperparâmetros contínuos

Quando o interesse é apenas nos hiperparâmetros contínuos ou discretos ordinais — taxa de aprendizado, momentum, weight decay, dropout rate, tamanho do batch — o PSO é uma alternativa natural à BO, especialmente para espaços de dimensão moderada ($n \leq 10$ hiperparâmetros).

Cada partícula representa um vetor de hiperparâmetros $\theta \in \mathbb{R}^n$; a função de fitness é a acurácia de validação após treinamento com θ. O PSO opera exatamente como descrito na Aula 6, sem modificações — a única diferença é que $f(\boldsymbol{x})$ agora é o resultado de um processo de treinamento de rede neural.

Exemplo

Otimização de hiperparâmetros de uma MLP para classificação (dataset MNIST, 10 000 amostras de treino, 2 000 de validação). Espaço: $\eta \in \left[10^{\{-4\}}, 10^{\{-1\}}\right]$ (log-scale), dropout $\in [0.0, 0.5]$, $n_{\text{hidden}} \in \{32, 64, 128, 256\}$, $n_{\text{camadas}} \in \{1, 2, 3, 4\}$.

Método	Acurácia val. (média)	Avaliações	Melhor θ encontrado
Manual (default Adam)	97.1%	1	$\eta = 10^{\{-3\}}$, dropout=0.0, 2 camadas×128
Grid Search (4^4)	97.8%	256	Impraticável em prod.
Random Search	97.9%	50	Boa cobertura
PSO ($N_p = 20$, 10 iter.)	98.2%	200	Explora interações
BO (GP-EI)	98.4%	50	Mais eficiente

Table 2: Comparação de métodos de otimização de hiperparâmetros para MLP no MNIST.

O PSO supera o random search ao explorar interações entre hiperparâmetros — a memória social $\boldsymbol{g}$ orienta o enxame para regiões do espaço onde combinações de dropout e η produzem boa

> generalização. A BO é mais eficiente por avaliação, mas o PSO é trivialmente paralelizável: cada partícula treina independentemente, o que o torna competitivo em ambientes com múltiplas GPUs.

5.5 NEAT: coevoluindo pesos e topologia

O **NEAT** (*NeuroEvolution of Augmenting Topologies*, Stanley & Miikkulainen, 2002) é o algoritmo de neuroevolução mais influente. Suas contribuições principais são:

Números históricos (*historical markings*): cada gene (conexão) recebe um número de inovação global que permite o cruzamento entre redes de topologias diferentes — resolvendo o problema de correspondência (*competing conventions*) que inviabilizava a neuroevolução antes do NEAT.

Especiação: redes com topologias similares competem apenas entre si, protegendo inovações topológicas recentes de extinção antes que possam ser otimizadas. A distância entre dois genomas é:

$$\delta = \frac{c_1 E + c_2 D}{N} + c_3 \overline{W}$$

onde E é o número de genes em excesso, D o número de genes disjuntos, $\overline{W}$ a diferença média de pesos, e c_1, c_2, c_3 são coeficientes de compatibilidade.

Complexificação incremental: começa com redes mínimas (sem camadas ocultas) e adiciona neurônios e conexões por mutação. Isso evita o sobreajuste topológico prematuro.

Aspecto	AG para NAS	PSO para HP	NEAT
O que otimiza	Arquitetura (discreta)	Hiperparâmetros (contínuos)	Pesos + topologia
Representação	Lista de camadas	Vetor real	Grafo de genes
Gradiente usado?	Não (só para treino proxy)	Não	Não
Ponto forte	Espaços discretos grandes	Eficiente, paralelizável	Complexidade adaptativa
Ponto fraco	Custo de avaliação alto	Discreto mal tratado	Escala mal para redes profundas
Aplicação típica	NAS em espaços moderados	Fine-tuning de hiperparâm.	Problemas de controle, RL

Table 3: Comparação das abordagens metaheurísticas para otimização de redes neurais.

5.6 Posicionamento: metaheurísticas vs. NAS baseado em gradiente

O NAS moderno — DARTS (Liu et al., 2019), ENAS (Pham et al., 2018) — usa **gradiente arquitetural** para otimizar a arquitetura de forma diferenciável, tratando a escolha de operadores como variáveis contínuas relaxadas. Isso é 10–100× mais rápido que métodos evolutivos em benchmarks padrão.

Por que então usar metaheurísticas para NAS?

- **Espaços não-diferenciáveis**: quando a arquitetura inclui decisões verdadeiramente discretas que não admitem relaxação contínua (e.g., escolha de tipo de operação em circuitos quânticos, estrutura de árvore de decisão).

- **Funções de fitness complexas**: quando o objetivo inclui múltiplos critérios não-diferenciáveis (acurácia + latência de inferência + tamanho do modelo) — domínio natural do NSGA-II aplicado a NAS multiobjetivo.
- **Sem acesso ao gradiente**: caixas-pretas (APIs de modelos, simuladores externos, avaliação humana).
- **Robustez a mínimos locais**: o gradiente arquitetural pode ficar preso; métodos populacionais mantêm diversidade.

> **Observação**
> A combinação mais eficaz na prática atual é **híbrida**: usa-se BO ou random search para explorar o espaço de hiperparâmetros de forma eficiente, e treinamento por gradiente (Adam, SGD) para otimizar os pesos dado cada conjunto de hiperparâmetros. Metaheurísticas e gradiente não competem — são complementares em diferentes níveis da hierarquia de otimização.

6 Síntese do Curso

Encerramos o curso com uma perspectiva integradora. Os cinco módulos construíram uma progressão coerente:

Módulo 1 estabeleceu o vocabulário e a epistemologia: formulação matemática de problemas de otimização, o Teorema No Free Lunch como fundamento da humildade algorítmica, e metodologia experimental rigorosa como base de toda comparação válida. O Problema da Mochila 0-1 foi introduzido como fio condutor.

Módulo 2 introduziu as metaheurísticas de trajetória — Hill Climbing, SA e Busca Tabu — no TSP com $n = 30$ cidades. A progressão mostrou como memória e controle de temperatura permitem superar ótimos locais: HC estagna, SA escapa com probabilidade controlada, BT escapa com memória determinística.

Módulo 3 expandiu para o paradigma populacional: AG e DE avaliados nas funções benchmark CEC, PSO comparado em benchmarks contínuos, ACO aplicado ao TSP como domínio histórico e natural. A síntese do módulo mapeou os quatro algoritmos em classes de problema por espaço natural de busca e tipo de memória.

Módulo 4 abordou otimização multiobjetivo: dominância de Pareto, NSGA-II validado nas funções ZDT ($m = 2$), NSGA-III e MOEA/D validados nas funções DTLZ ($m = 3, 5$). A comparação com escalarização demonstrou a limitação matemática fundamental da soma ponderada em fronteiras descontínuas (ZDT3) e a necessidade de métodos populacionais para muitos objetivos (DTLZ com $m = 5$).

Módulo 5 fechou com tópicos avançados: calibração automática por BO, análise de paisagem como ferramenta diagnóstica pré-algoritmo, os limites do paradigma (quando usar métodos exatos), e a integração com ML — culminando na neuroevolução como aplicação onde metaheurísticas e gradiente são complementares.

O fio condutor foi sempre o mesmo: **metaheurísticas são ferramentas, não oráculos**. Sua eficácia depende de entender a estrutura do problema, projetar representações e operadores adequados, calibrar parâmetros com rigor, e comparar resultados com metodologia estatística honesta. O engenheiro que domina esses princípios é capaz de avaliar e aplicar qualquer nova metaheurística que aparecer na literatura — porque entende o que está por baixo da inspiração biológica ou física de cada proposta.

7 Exercícios Finais

Exercício

1 (Otimização Bayesiana). Usando a biblioteca `Optuna` (Python), implemente a calibração automática do SA para o TSP com $n = 30$ cidades (instância das Aulas 3–4). Espaço: $T_0 \in [100, 5000]$, $\alpha \in [0.88, 0.99]$, $L \in [50, 500]$. Métrica: comprimento médio do melhor tour em 20 execuções. (a) Compare BO (50 trials) com random search (50 trials) em termos do melhor resultado e número de trials até convergência. (b) Plote a superfície de $\mathcal{Q}(T_0, \alpha)$ para L fixo no valor ótimo encontrado — a superfície é suave? Isso justifica o uso de GP como surrogate?

Exercício

2 (Análise de paisagem). Para o TSP com $n = 30$ cidades: (a) execute 500 caminhadas aleatórias 2-opt de comprimento 100 e calcule o comprimento de correlação ℓ; (b) execute Hill Climbing 2-opt a partir de 300 pontos aleatórios e construa um histograma dos ótimos locais encontrados — quantos ótimos locais distintos existem (tolerância $\varepsilon = 0.5\%$ do comprimento)? (c) com base em ℓ e no número de ótimos locais, o resultado é consistente com o desempenho relativo de SA e BT observado na Aula 4?

Exercício

3 (Quando não usar). Para cada problema abaixo, justifique se você usaria metaheurística ou método exato/específico, citando o critério decisivo: (a) Knapsack 0-1 com $n = 15$ itens e $W = 50$; (b) problema de atribuição de 8 tarefas a 8 máquinas com custo linear; (c) Knapsack quadrático com $n = 100$ itens e interações não-convexas entre pares; (d) TSP com $n = 12$ cidades; (e) minimização de função $f(x) = (x - 3)^2 + 2$, $x \in [-10, 10]$.

Exercício

4 (NAS com AG). Implemente um AG simples para NAS em uma MLP para classificação binária no dataset *breast cancer* do scikit-learn. Espaço de arquiteturas: 1–4 camadas densas, cada uma com $\{32, 64, 128, 256\}$ neurônios e ativação $\{\text{ReLU}, \tanh\}$. Fitness: acurácia de validação após 20 épocas de treinamento (Adam, $\eta = 10^{\{-3\}}$). (a) Use $N_{\text{pop}} = 10$, $T_{\max} = 20$ gerações, cruzamento de um ponto, mutação bit-a-bit com $p_m = 0.1$. (b) Compare a melhor arquitetura encontrada com random search de 200 avaliações e com a MLP padrão do scikit-learn. (c) Plote a curva de convergência do fitness máximo por geração. O AG encontra arquiteturas melhores que o random search com o mesmo orçamento de 200 avaliações?

Exercício

5 (Projeto integrador). Escolha um problema de otimização de sua área de interesse. (a) Formule-o matematicamente (variáveis, objetivos, restrições). (b) Classifique-o segundo a taxonomia da Aula 1. (c) Analise a paisagem com uma amostra de 500 soluções aleatórias (autocorrelação ou FDC). (d) Com base nos resultados, justifique a escolha de um algoritmo do curso. (e) Implemente o algoritmo escolhido, calibre os hiperparâmetros com random search (30 configurações), e reporte os resultados com metodologia completa: 30 execuções, custo médio $\pm$ desvio padrão, curva de convergência, comparação com ao menos um método de referência.

Leituras recomendadas:

• Bergstra, J., Bengio, Y. Random search for hyper-parameter optimization. **Journal of Machine Learning Research**, 13, 281–305, 2012.
• Shahriari, B. et al. Taking the human out of the loop: A review of Bayesian optimization. **Proceedings of the IEEE**, 104(1), 148–175, 2016.
• Stanley, K., Miikkulainen, R. Evolving neural networks through augmenting topologies. **Evolutionary Computation**, 10(2), 99–127, 2002. *(NEAT — leitura obrigatória para neuroevolução)*
• Real, E. et al. Regularized evolution for image classifier architecture search. **Proc. AAAI**, 4780–4789, 2019. *(AG para NAS em larga escala)*
• Liu, H., Simonyan, K., Yang, Y. DARTS: Differentiable architecture search. **Proc. ICLR**, 2019. *(NAS baseado em gradiente — contexto comparativo)*
• Ochoa, G. et al. Mapping the global structure of TSP fitness landscapes. **Journal of Heuristics**, 14(4), 337–351, 2008.
• Jin, Y. Surrogate-assisted evolutionary computation: Recent advances and future challenges. **Swarm and Evolutionary Computation**, 1(2), 61–70, 2011.
• Talbi, E.-G. **Metaheuristics: From Design to Implementation**. Wiley, 2009. Caps. 6–8.